

Clausole di Horn

Horn clauses, forward and backward chaining

- In molti contesti pratici sono utilizzate clausole dalla forma molto specifica, per le quali sono stati studiati meccanismi di inferenza ad hoc:
 - Clausole di Horn
 - Forward Chaining e Backward Chaining

Horn Clauses (clausole di Horn)

- **Definizione:** una **clausola di Horn** è una disgiunzione di letterali di cui al più uno è positivo
 - Se la clausola contiene esattamente un letterale positivo è detta **clausola definita**
 - **Esempi:** $\neg B \vee C$ oppure $\neg A \vee \neg B \vee C$ oppure $\neg A \vee \neg B$ sono clausole di Horn, le prime due sono anche clausole definite
- Catturano delle implicazioni in cui la formula implicante è una congiunzione di letterali positivi e la formula implicata è un singolo letterale positivo, esempi:
 $B \Rightarrow C$ oppure $A \wedge B \Rightarrow C$
- Costituiscono la base della programmazione logica

Clausole di Horn: vantaggi

- Su clausole di Horn è possibile applicare meccanismi di inferenza molto **naturali per gli esseri umani**
- Consentono di verificare la consequenzialità logica in un **tempo che cresce linearmente con la dimensione della KB** (quindi l'inferenza nel caso proposizionale è computazionalmente economica)

Forward Chaining (concatenazione in avanti)

- Permette di derivare una query data da un singolo simbolo proposizionale da una KB costituita da clausole di Horn
- Procedimento iterativo, guidato dai dati:
 - 1) Si parte dai fatti conosciuti
 - 2) Si applica il modus ponens (da $F \Rightarrow Q$ e F derivo Q) , ragionamento deduttivo
 - 3) Se tutte le premesse di un'implicazione sono vere, si aggiunge il letterale implicato all'insieme dei fatti conosciuti
 - 4) Terminazione: o si ottiene la query (return true) o a un certo punto non si potranno fare altre inferenze (return false)
- Ha complessità lineare nella dimensione della KB

Esempio

La figura rappresenta la seguente KB sotto forma di grafo AND-OR. Gli archi uniti da un archetto rappresentano letterali in AND, le frecce rappresentano degli OR dati da clausole aventi come testa lo stesso letterale

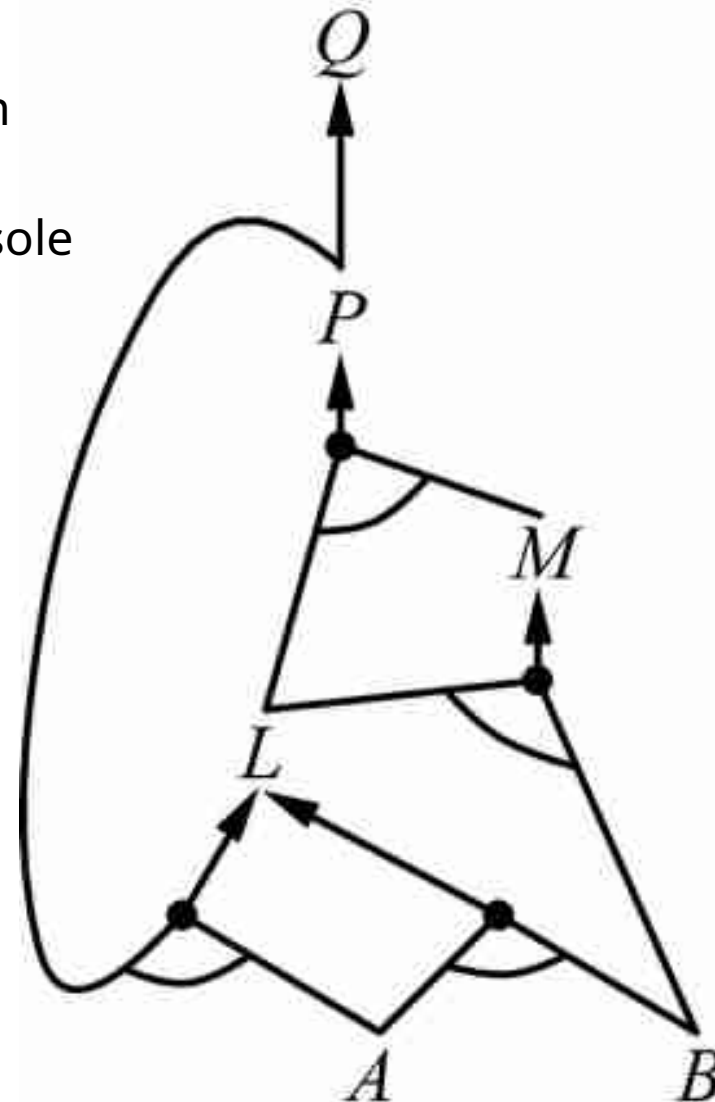
$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$



Esempio

Partendo dai nodi attivati direttamente dai fatti
l'inferenza si propaga: un arco AND si attiva
quando tutti i suoi congiunti sono veri;
un arco OR quando uno dei disgiunti è vero.

$$P \Rightarrow Q$$

$$L \wedge M \Rightarrow P$$

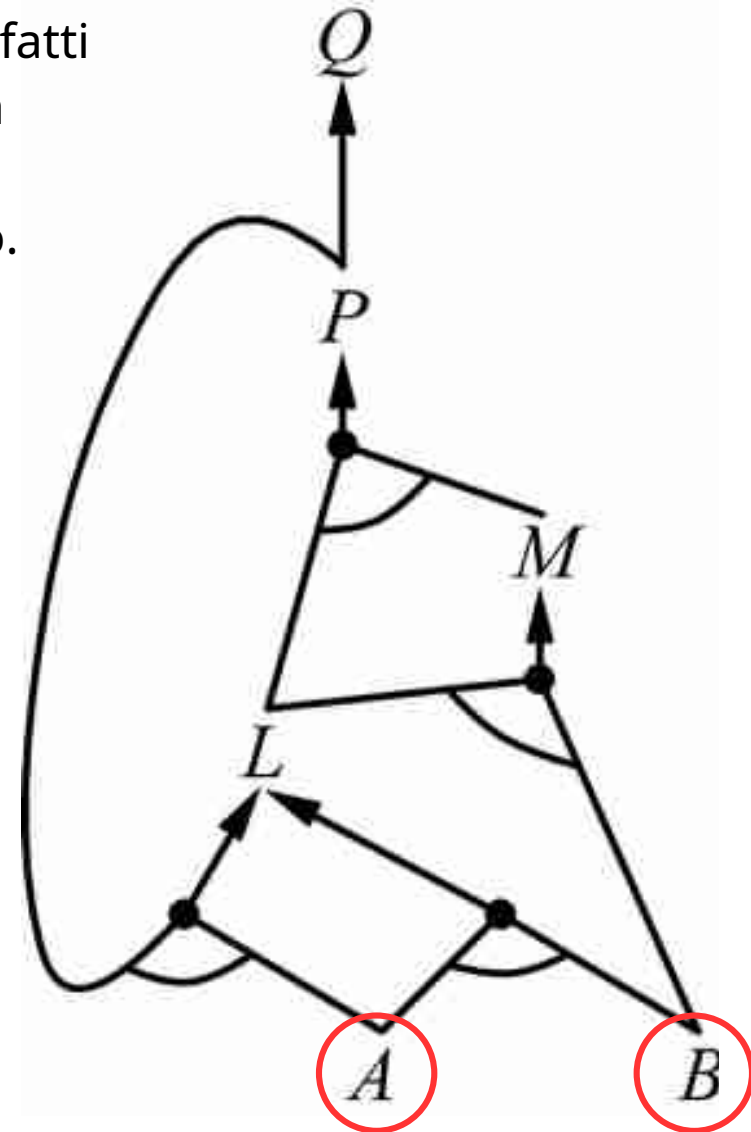
$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

Fatti: A, B

Si vuole dimostrare Q



Esempio

I cerchi rossi evidenziano alcuni letterali che risulteranno veri con questo procedimento:
A e B perché attivati dai fatti, L perché A e B in and costituiscono l'antecedente di una regola che ha L come conseguente, ecc.

$$P \Rightarrow Q$$

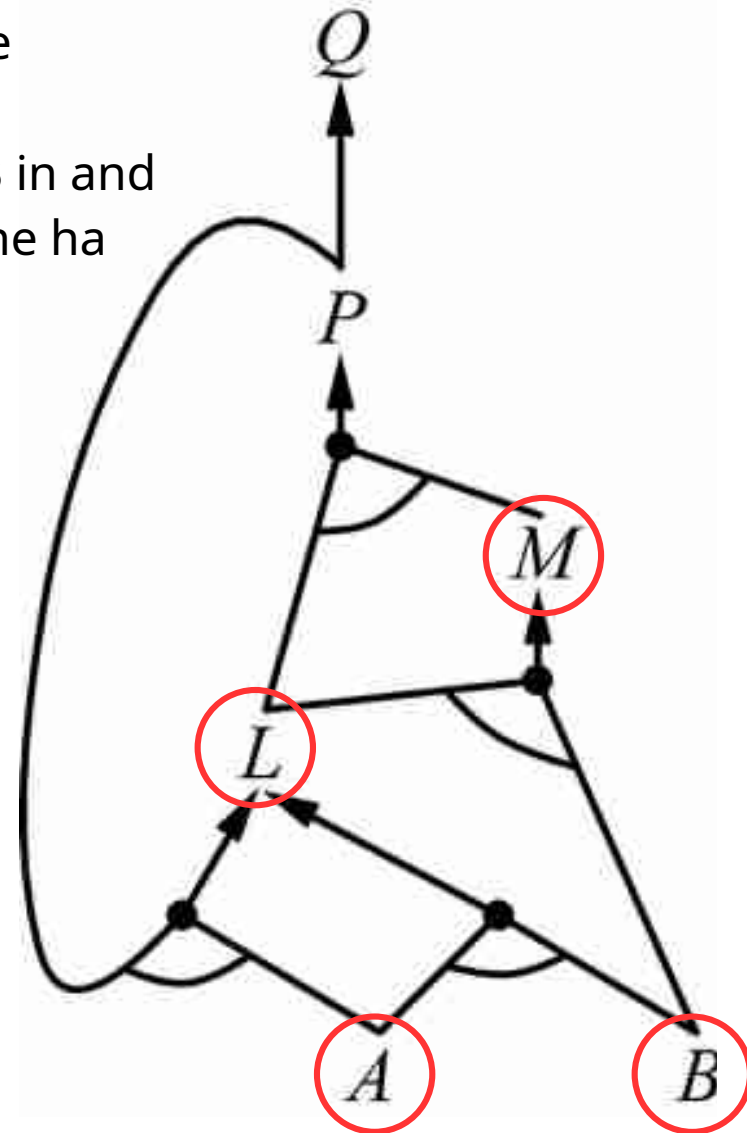
$$L \wedge M \Rightarrow P$$

$$B \wedge L \Rightarrow M$$

$$A \wedge P \Rightarrow L$$

$$A \wedge B \Rightarrow L$$

Fatti: A, B



Commenti sul forward chaining

- **Complessità lineare**
- **Completo**: permette di derivare tutte le formule atomiche dimostrabili a partire dalla KB
- **Inconscio**: è guidato dai dati e non usa l'informazione relativa al goal (la formula che stiamo cercando di dimostrare)
- È adeguato a risolvere problemi come per esempio il riconoscimento di oggetti
- Può attivare molte **inferenze inutili** ai fini della dimostrazione della formula in oggetto

Backward chaining (concatenazione all'indietro)

- Parte dalla formula da dimostrare (**goal**):
 - Se risulta già vera termina restituendo true
 - Altrimenti cerca clausole di Horn di cui la formula è conclusione e cerca di dimostrarne le premesse usando come informazione aggiuntiva i fatti noti

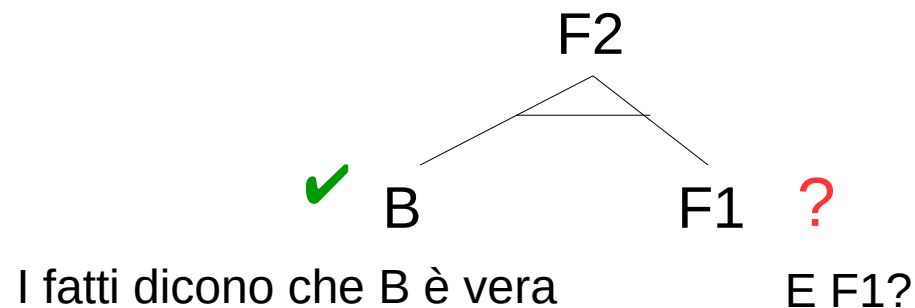
Backward chaining

Supponiamo di avere:

R1) $A \wedge C \Rightarrow F1$

R2) $B \wedge F1 \Rightarrow F2$

E di voler dimostrare che dati A, B e C, F2 è vera. F2 non appartiene ai fatti noti ma abbiamo R2



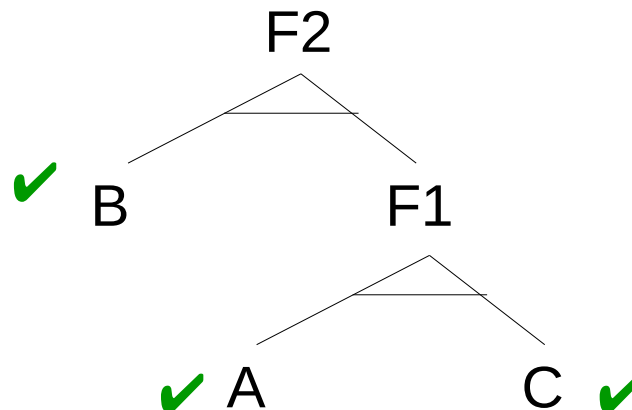
Backward chaining

Supponiamo di avere:

R1) $A \wedge C \Rightarrow F1$

R2) $B \wedge F1 \Rightarrow F2$

... F1 non appartiene ai fatti noti ma abbiamo R1, che ci dice che F1 è vera quando A e C sono veri. I fatti dicono che A e C sono veri, F2 è vera



I fatti dicono che B è vera

Esempio

Stessa KB, fatti e obiettivo di prima

Si sfruttano due informazioni:

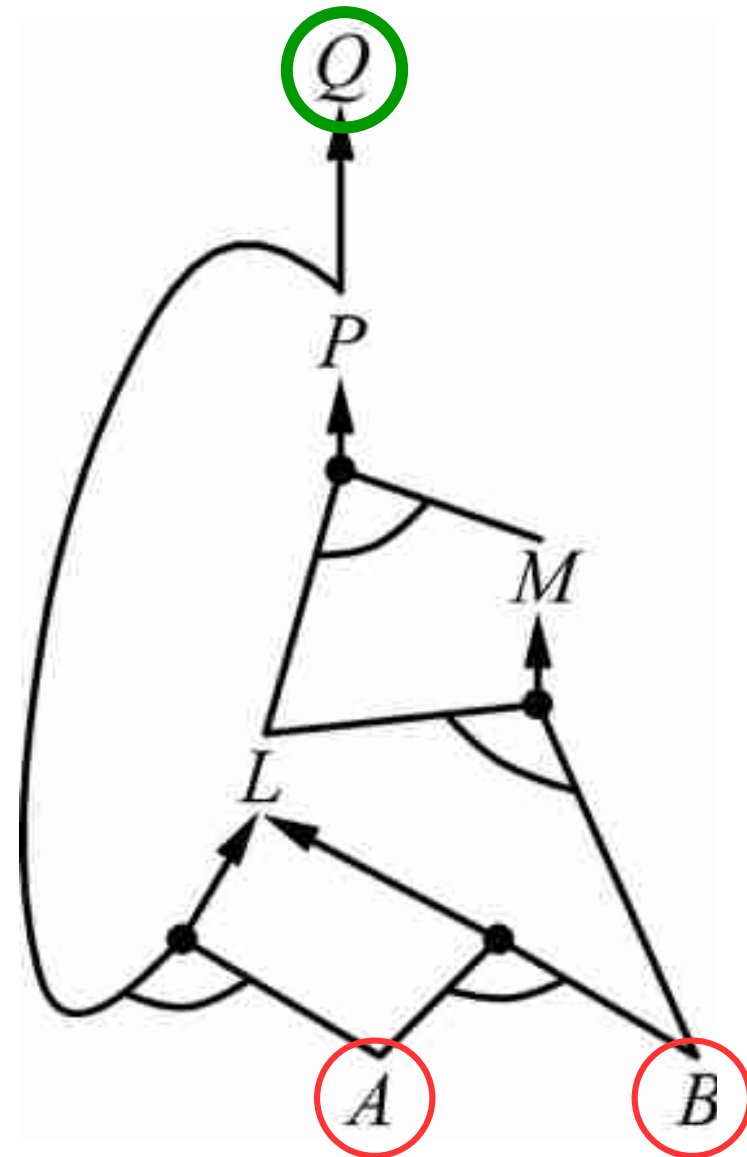
- Una è costituita dall'obiettivo
- L'altra è costituita dai fatti

Nella ricerca:

- Evitare i loop
- Se un sottogoal è già stato dimostrato, non dimostrarlo di nuovo

BC:

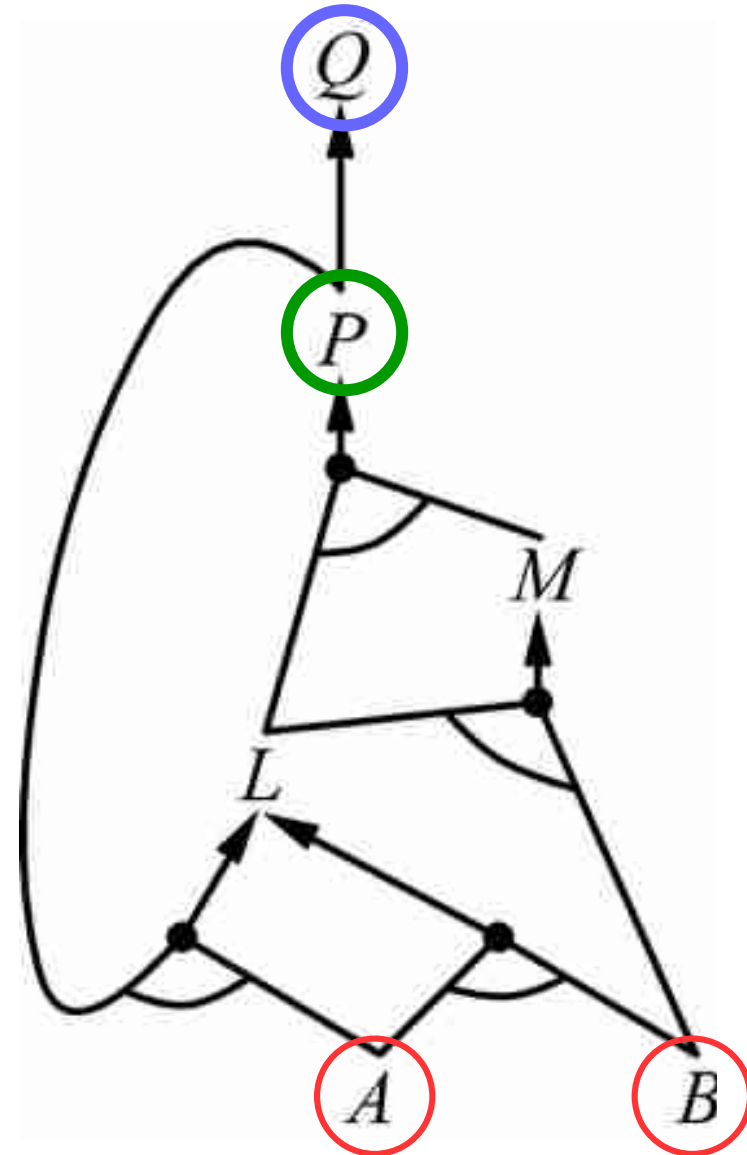
- Q è vera se P è vera ...



Esempio

BC:

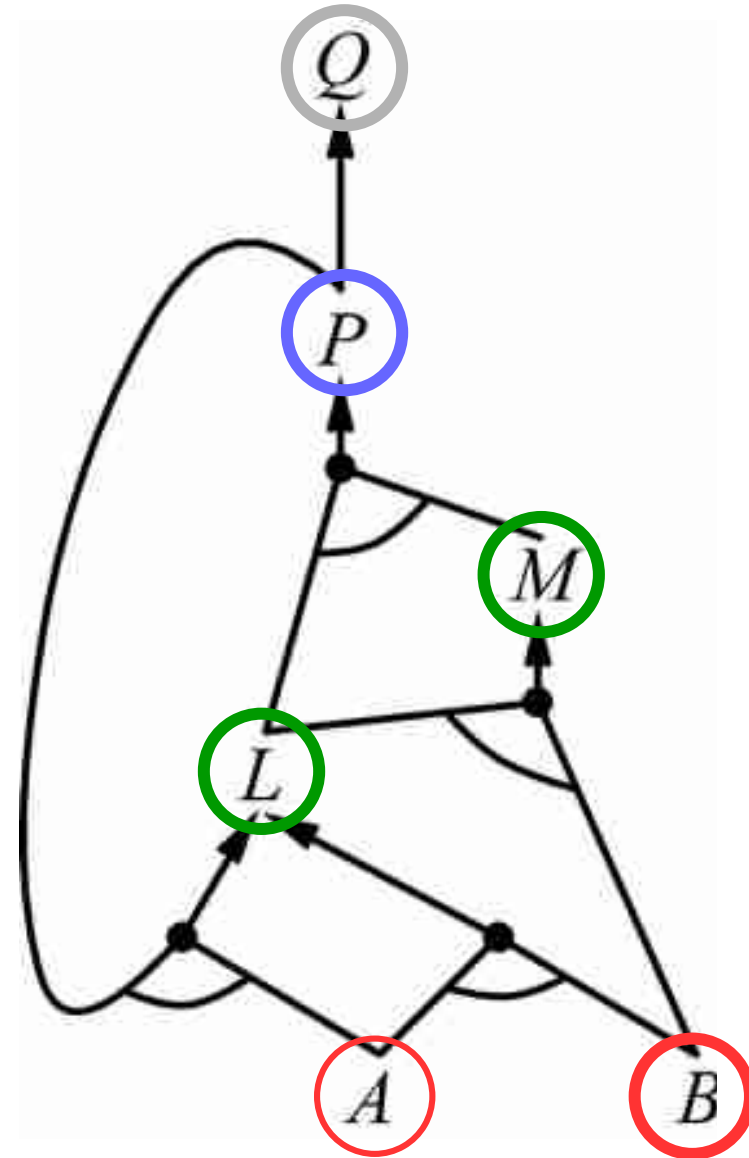
- P è vera se L e M sono vere



Esempio

BC:

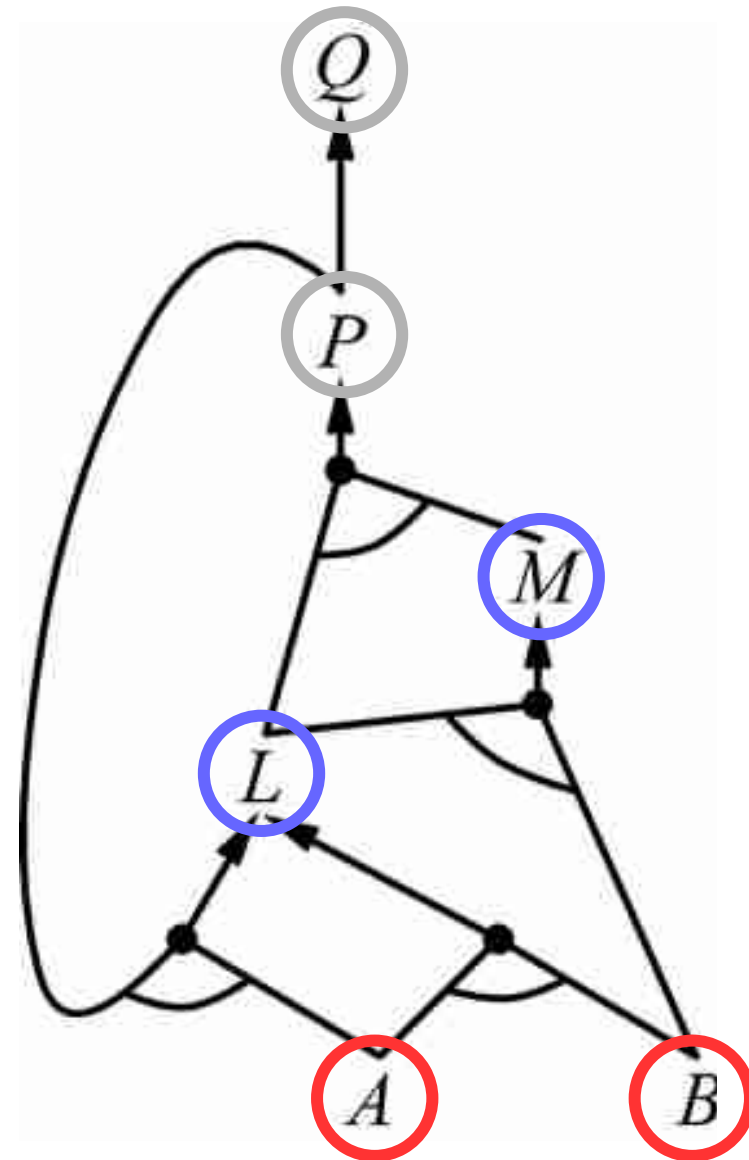
- P è vera se L e M sono vere
- M è vera se L e B sono vere:
 B è un fatto, L è da dimostrare



Esempio

BC:

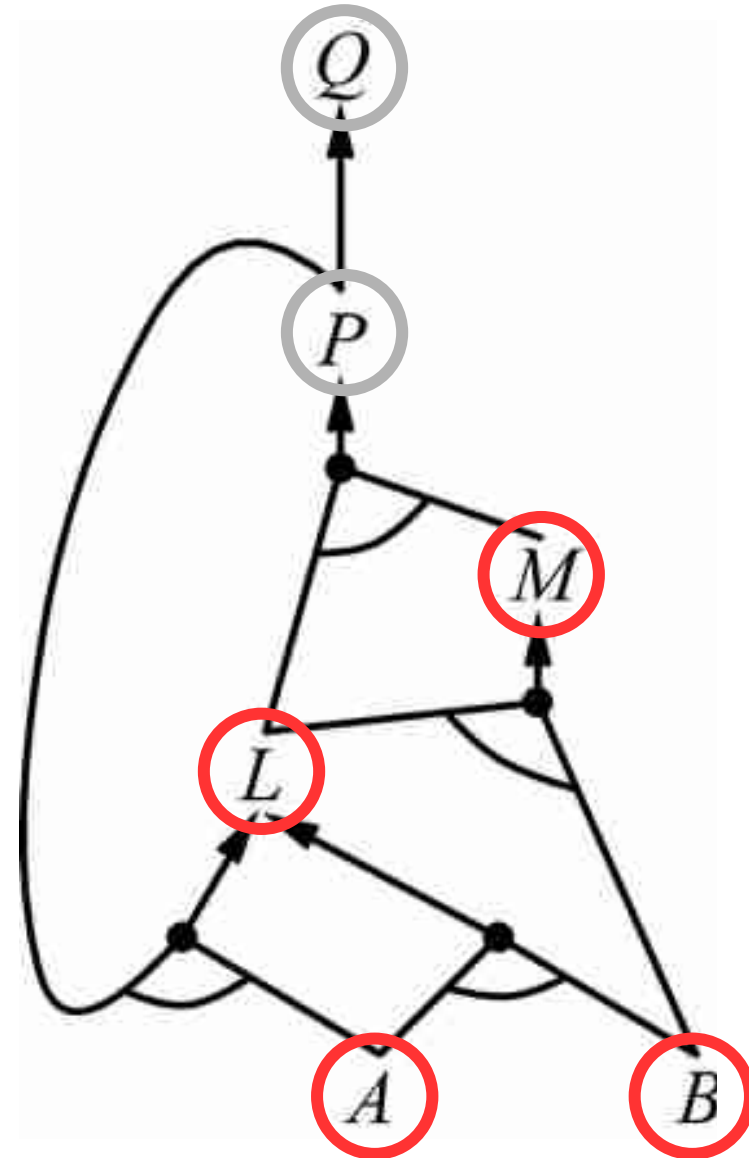
- P è vera se L e M sono vere
- M è vera se L e B sono vere:
B è un fatto, L è da dimostrare
- L è vera se A e B sono vere:
A e B sono fatti!
L è vera anche quando A e P
sono vere ma di P non conosciamo
il valore di verità e comunque ci
basta che una delle due regole sia
vera



Esempio

BC:

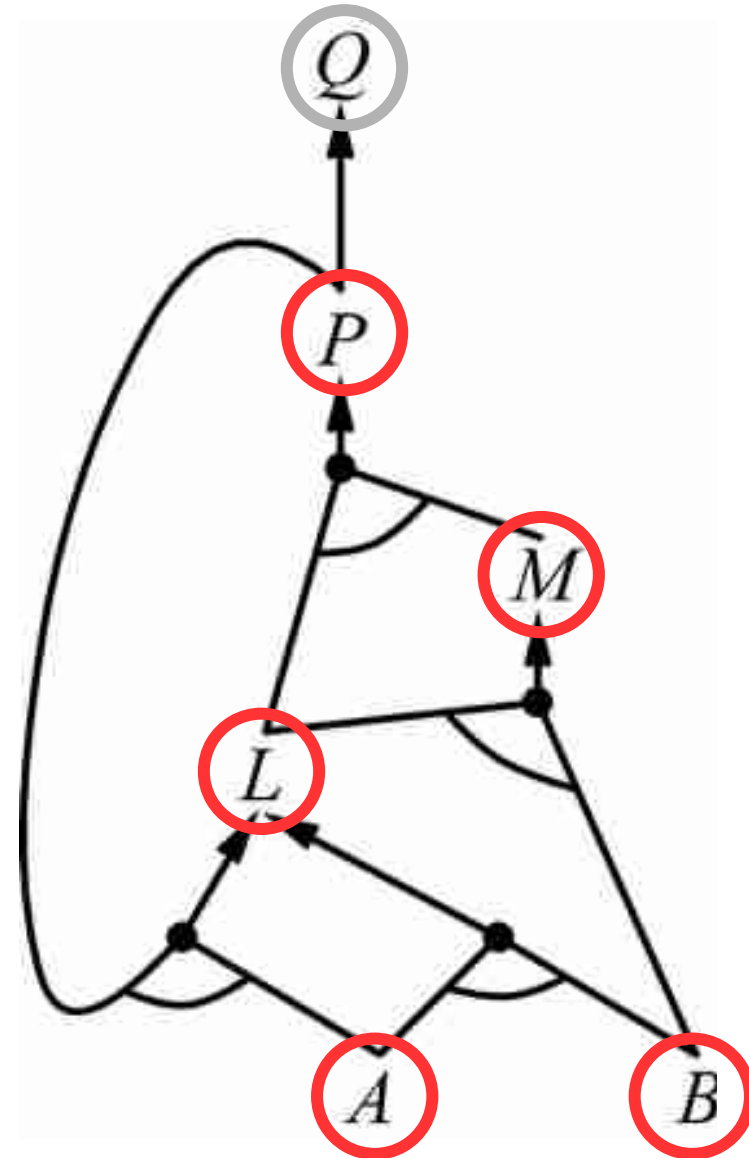
- I valori di verità si propagano dal basso verso l'alto (come valori di ritorno)



Esempio

BC:

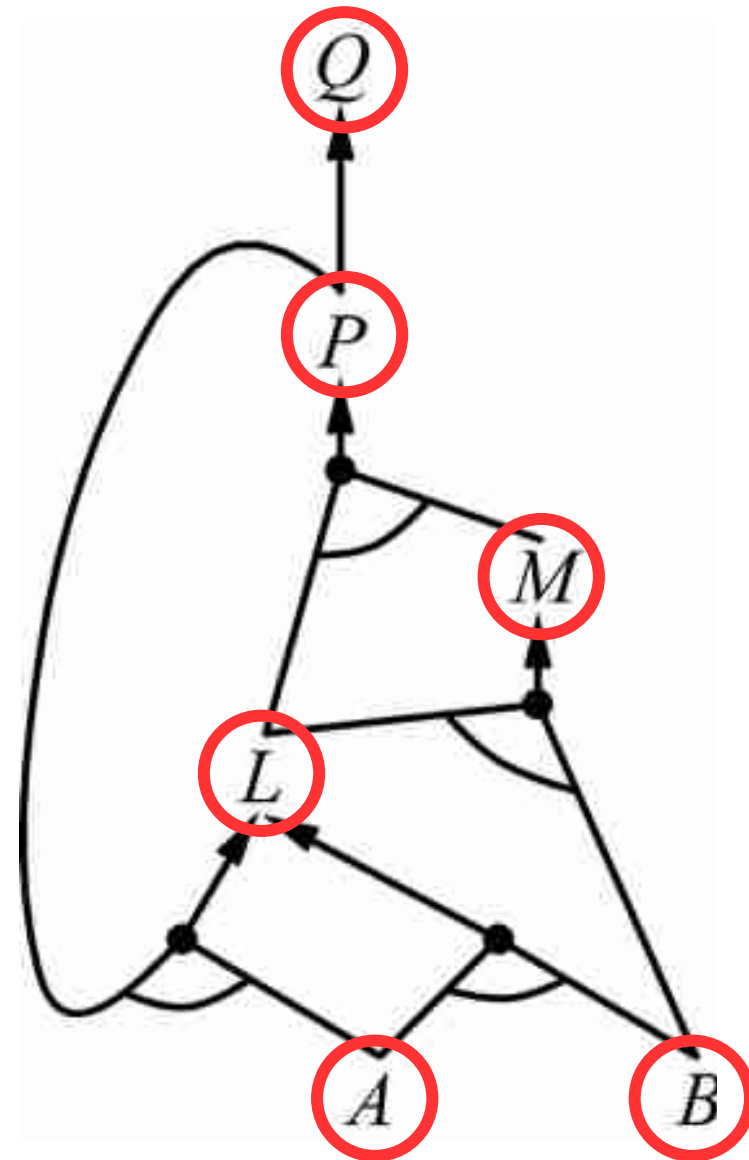
- P risulterà vera



Esempio

BC:

- E infine anche Q risulterà vera
- Il loop da P a L sarà ignorato



Commento su backward chaining

- Realizza una forma di ragionamento **guidato dagli obiettivi**
- È usato nel **theorem proving** e nella **programmazione logica** come meccanismo di inferenza
- Spesso è **più efficiente** del forward chaining in quanto l'uso del goal focalizza la ricerca
- La complessità temporale è **meno che lineare**